

PASOS PARA SYMPONY

1. Preparar el entorno

Antes de desplegar, asegúrate de que tu servidor esté listo para correr una aplicación Symfony. Asegúrate de tener las siguientes dependencias instaladas:

- **PHP** (versión compatible con tu proyecto Symfony).
- **Composer** (para manejar las dependencias de PHP).
- **Base de datos** (MySQL, PostgreSQL, etc. según sea el caso).
- **Servidor web** como **NGINX** o **Apache**.
- **Servidores opcionales** como **Redis**, **Varnish**, si estás usando cache avanzado.

2. Preparar la aplicación

Antes de subir tu aplicación al servidor, debes asegurarte de que el código esté listo para el entorno de producción. Aquí están los pasos esenciales:

a. Configurar las variables de entorno

Symfony usa el archivo `.env` para definir las variables de entorno. En producción, generalmente se usan variables diferentes a las de desarrollo, así que necesitarás un archivo `.env.local` o directamente configurar estas variables en tu servidor.

Ejemplo de variables de entorno para producción:

```
APP_ENV=prod
APP_DEBUG=0
DATABASE_URL="mysql://user:password@127.0.0.1:3306/dbname"
```

b. Instalar las dependencias

En el servidor, corre el siguiente comando para instalar las dependencias con **Composer**:

```
composer install --no-dev --optimize-autoloader
```

El parámetro `--no-dev` asegura que las dependencias de desarrollo no se instalen, y `--optimize-autoloader` optimiza el autoloading de clases para producción.

c. Compilar los assets

Si estás utilizando **Webpack Encore** o algún sistema de compilación para los assets, debes compilar los assets para producción:

```
php bin/console assets:install --symlink  
npm run production
```

Esto generará los archivos minificados y listos para producción.

3. Caching y optimización

Para garantizar un buen rendimiento en producción, debes hacer que Symfony optimice sus archivos de caché y configuración:

```
php bin/console cache:clear --env=prod --no-debug  
php bin/console cache:warmup --env=prod
```

Esto asegura que el caché esté listo y que la aplicación se cargue rápidamente. Hacerlo antes de subir la aplicación.

4. Base de datos

a. Configuración de la base de datos

Configura la base de datos en tu servidor de producción. Si ya tienes datos en producción, asegúrate de que el archivo `.env` apunte a la base de datos correcta.

b. Migraciones

Si necesitas ejecutar migraciones de base de datos, usa el siguiente comando:

```
php bin/console doctrine:migrations:migrate --no-interaction
```

5. Subir el proyecto y la base de datos.

Sube con FTP la carpeta de tu proyecto en producción al servidor.

La ruta será:

```
http://url_del_servidor/public/index.php
```

Ejecuta el script exportado de tu base de datos importándolo en la base de datos del servidor.

6. Configurar el servidor web

a. NGINX

Si estás usando NGINX, puedes crear un bloque de servidor como el siguiente:

```
server {
    listen 80;
    server_name example.com;
    root /var/www/symfony/public;

    index index.php index.html index.htm;

    location / {
        try_files $uri /index.php$is_args$args;
    }

    location ~ ^/index\.php(/|$) {
        fastcgi_pass unix:/var/run/php/php7.4-fpm.sock; # Asegúrate de que esté configurado
según tu versión de PHP
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php7.4-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

b. Apache

Si estás usando Apache, puedes utilizar el siguiente archivo `.htaccess` en el directorio `public/`:

```
<IfModule mod_rewrite.c>
  RewriteEngine On

  # Redirect to front controller
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule ^(.*)$ /index.php [QSA,L]
</IfModule>
```

Asegúrate de que Apache tenga habilitados los módulos `mod_rewrite` y `mod_php`.

7. Configuración de permisos

Asegúrate de que los directorios `var/` y `public/` tengan los permisos adecuados para que Symfony pueda escribir en ellos:

```
chmod -R 777 var/
chmod -R 777 public/
```

Asegúrate de que los archivos de caché y logs se puedan escribir correctamente.

8. Seguridad

Asegúrate de que los archivos sensibles, como el `.env` o cualquier archivo de configuración privado, no estén accesibles desde el navegador. Los servidores como NGINX y Apache deberían estar configurados para prevenir el acceso a estos archivos.

9. Verificación y monitoreo

Una vez que hayas desplegado tu aplicación, verifica que todo esté funcionando correctamente accediendo a la URL de tu servidor. También es recomendable configurar monitoreo (con herramientas como **New Relic**, **Sentry**, **Datadog**, etc.) para poder detectar errores y problemas de rendimiento.